Statistical Methods in Artificial Intelligence

Assignment 2: Applied Deep Learning & Sequential Modeling

International Institute of Information Technology, Hyderabad

Spring 2026 | Module 2 – Neural Networks

Total Points:	215
Estimated Effort:	24–28 Hours
Compute:	Minimum 4 GB VRAM (laptop GPU) *or* Google Colab Free Tier
Submission:	Jupyter Notebook (.ipynb) + PDF report via Course Portal

---

Important

1. All code must be your own. You may use PyTorch, torchvision, and standard scientific-Python libraries. Pre-trained weights are **not** allowed unless explicitly stated.
2. Theory answers must be in your own words; copy-pasted ChatGPT / Gemini dumps will receive **zero marks** and will be reported to the academic integrity committee.
3. Every plot must have labelled axes, a legend, and a title. Unlabelled plots will lose 50% of their marks.
4. Seeds: set `torch.manual_seed(42)` and `np.random.seed(42)` for reproducibility.
5. Late submissions will not be accepted.

Note

Each question (Q1–Q3) must be submitted as a **single main file**—either a Jupyter Notebook (`.ipynb`) or a PDF (`.pdf`)—containing all code, outputs (printed results, plots), and written explanations for that question. You may create additional helper Python files (`.py`) for utility functions or model definitions and import them in your notebook; include these helper files in your submission.
Question 4 (Concept Deep-Dive) must be submitted as a standalone PDF via the course portal.

File naming convention: Prefix every file with `Qi_` where `i` is the question number. The main answer file for each question must be named exactly:

`Q1.ipynb` or `Q1.pdf`
`Q2.ipynb` or `Q2.pdf`
`Q3.ipynb` or `Q3.pdf`
`Q4.pdf`

Helper files should follow the pattern `Qi_<descriptive_name>.py` (e.g., `Q2_model.py`, `Q3_attention_utils.py`). Ensure all file names are **semantically meaningful**—do not use names like `untitled.ipynb` or `code.py`. Submissions with incorrect naming will incur a **2-mark penalty** per question.

Question 1: The Mechanics of Learning – Optimizers, Normalization, and Memory

Estimated Time: 7 Hours **Total: 55 Points**

Objective: Understand how optimizers navigate the loss landscape, why normalization layers are critical, and how to train models on hardware-constrained settings.

Dataset: CIFAR-10 (via `torchvision.datasets`).

Part A: Theoretical Foundations [25 Points]

1A.1 Optimizer Dynamics (5 pts). The standard SGD update is $w_{t+1} = w_t - \eta \nabla L(w_t)$.

(a) Write the modified update equation when a momentum term β is added. Explain, in your own words, how the velocity term helps the optimizer escape shallow local minima and reduces oscillation in narrow ravines of the loss surface.

(b) In a *single sentence*, explain why Nesterov momentum is sometimes called "look-ahead" momentum.

1A.2 Adaptive vs. Fixed Learning Rates (5 pts).

(a) Adam maintains per-parameter estimates of the first moment (mean) and second moment (uncentered variance) of gradients. Explain intuitively why this gives Adam a warm-start advantage over vanilla SGD in the first few epochs.

(b) Practitioners training ImageNet-scale classifiers often switch from Adam to SGD-with-momentum after an initial phase. Give one concrete reason related to the *generalization gap* between Adam and SGD observed in the literature.

1A.3 Why Batch Normalization? (8 pts).

This sub-question builds intuition for *why* normalization matters inside deep networks.

(a) **Internal Covariate Shift (3 pts).** Consider a 10-layer MLP where every layer doubles its input magnitude (i.e., each pre-activation is roughly $2\times$ the previous layer's). After 10 layers, by what factor has the magnitude grown? Explain why this "exploding activation" phenomenon makes choosing a good learning rate extremely difficult.

(b) **Mechanics of BatchNorm (3 pts).** Write the BatchNorm transform for a mini-batch $\{x_1, \ldots, x_B\}$:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \qquad y_i = \gamma\, \hat{x}_i + \beta$$

Explain the role of γ and β. Why are they necessary—wouldn't normalizing alone suffice?

(c) **BatchNorm at test time (2 pts).** During inference, the mini-batch statistics $\mu_{\mathcal{B}}, \sigma_{\mathcal{B}}^2$ are unavailable (batch size might be 1). Explain what running statistics are used instead and how they are computed during training.

1A.4 Gradient Accumulation (4 pts). Your GPU can hold a batch of $B_{\text{small}} = 8$, but you want an effective batch of $B_{\text{large}} = 32$.

(a) Show mathematically that accumulating gradients over $N = 4$ forward–backward passes (each on 8 samples) and then calling `optimizer.step()` once yields the same parameter update as processing all 32 samples in a single pass.

(b) *Thought question:* Does gradient accumulation give the *exact* same BatchNorm statistics as a true batch of 32? Why or why not?

1A.5 Quick Conceptual Questions (3 pts, 1 pt each). Answer each in at most two sentences.

(a) Why is the bias term in a convolutional layer redundant when it is immediately followed by a BatchNorm layer?

(b) If you freeze the γ and β parameters of a BatchNorm layer (i.e., stop their gradients), does training still benefit from the normalization? Justify briefly.

(c) Weight decay in AdamW is applied directly to the weights, not to the gradient. Why is this called "decoupled" weight decay?

Hint

For Q1A.3(a), think about powers of 2. For Q1A.4(b), think carefully about what mini-batch the running mean/variance see.

Part B: Practical Implementation [30 Points]

Build a **shallow CNN** (at most 3 Conv2D layers + 1 fully connected layer) to classify CIFAR-10.

1B.1 Optimizer Face-off (10 pts). Train the *identical* architecture for 15 epochs using three optimizers:

- Vanilla SGD ($\eta = 0.01$)
- SGD with momentum ($\eta = 0.01,\ \beta = 0.9$)
- AdamW ($\eta = 0.001$, default betas)

Plot *training loss* and *validation accuracy* curves (all three on the same figure, two subplots). Write 3–4 sentences interpreting the differences.

1B.2 BatchNorm Ablation (8 pts). Take your best optimizer from 1B.1. Train two versions of your CNN:

- **With** BatchNorm after every Conv2D layer (before ReLU).
- **Without** any normalization.

Plot both training loss curves on the same graph. Report final validation accuracy for each. In 2–3 sentences, explain why the gap exists.

1B.3 Focal Loss (6 pts). Implement Focal Loss:

$$\mathrm{FL}(p_t) = -\alpha_t\,(1 - p_t)^{\gamma}\,\log(p_t), \qquad \gamma = 2.0$$

Train your (BatchNorm-equipped) CNN with Focal Loss and compare validation accuracy against standard Cross-Entropy. In 1–2 sentences, explain when Focal Loss provides a meaningful advantage over CE.

1B.4 Gradient Accumulation in PyTorch (6 pts). Hard-code `batch_size=8` in your DataLoader. Implement a training loop that accumulates gradients over 4 micro-batches before calling `optimizer.step()` (effective batch = 32). Print the loss for the first 50 optimizer steps. Verify the loss is decreasing.

Question 2: Vision in the Real World – Indian Currency Classification

Estimated Time: 8 Hours **Total: 75 Points**

Objective: Apply CNNs to a culturally relevant, noisy dataset. Learn to reason about spatial dimensions, design parameter-efficient architectures, and interpret model decisions.

Dataset: "Indian Currency Notes" (denominations INR 10 – INR 2000). Create your own dataset. Upload your dataset on google drive and share the link as a part of the submission. Resize all images to 128×128.

Note

If you cannot download the currency dataset, you may substitute the Oxford Flowers-102 dataset (also available via `torchvision`). State this clearly in your submission.

Part A: Theoretical Foundations [25 Points]

2A.1 Spatial Dimension Calculation (5 pts).

The output spatial dimension for a convolutional layer is:

$$O = \left\lfloor \frac{I - K + 2P}{S} \right\rfloor + 1$$

(a) Calculate the *exact* output volume (channels $\times$ height $\times$ width) when a $3 \times 128 \times 128$ image is passed through a Conv2D layer with 32 filters, 3×3 kernel, stride 2, padding 1.

(b) Now pass that output through a second Conv2D with 64 filters, 3×3 kernel, stride 1, padding 1. What is the resulting volume?

2A.2 Receptive Field (5 pts).

(a) Define "receptive field" of a neuron in a convolutional layer. Why is it important for a network to have a receptive field that covers a large portion of the input image?

(b) Calculate the receptive field of a neuron at the output of two stacked 3×3 convolutions (both stride 1, no pooling). Compare this to a single 5×5 convolution. Which uses fewer parameters?

2A.3 Parameter Efficiency (8 pts).

(a) Calculate the exact number of learnable parameters (weights + biases) for a standard Conv2D layer mapping 32 input channels to 64 output channels with a 3×3 kernel.

(b) Now calculate the parameters for the same input-to-output mapping using a **Depthwise Separable Convolution** (a 3×3 depthwise conv followed by a 1×1 pointwise conv). Include biases.

(c) Compute the compression ratio. Why do MobileNet-style architectures rely heavily on this factorization?

2A.4 BatchNorm vs. LayerNorm – Choosing the Right Normalization (7 pts).

This question tests your understanding of *when* different normalization strategies are appropriate.

(a) **BatchNorm axis (2 pts).** In a CNN with feature maps of shape (B, C, H, W), BatchNorm computes mean and variance over which axes? Draw a simple diagram or write the axes explicitly.

(b) **Small-batch failure mode (2 pts).** Suppose you are training with a batch size of 2 (e.g., due to very high-resolution medical images). Explain why BatchNorm's statistics become unreliable. Name one alternative normalization (other than LayerNorm) that is robust to small batch sizes.

(c) **Why not BatchNorm in Transformers? (3 pts).** In NLP Transformers, sequences in a batch have different lengths (even after padding). Explain why computing batch-level statistics across the sequence dimension is problematic and why LayerNorm (which normalizes across the feature dimension of each individual token) is the standard choice.

Part B: Practical Implementation [50 Points]

2B.1 Baseline CNN (10 pts).

Design a CNN with 4 convolutional blocks. Each block: `Conv2D` → `BatchNorm` → `ReLU` → `MaxPool(2)`. End with Global Average Pooling → a single Linear layer. Train for 15 epochs. Report training loss and test accuracy.

2B.2 Mobile-Friendly CNN (12 pts).

Create a second architecture where the *last two* convolutional blocks use Depthwise Separable Convolutions instead of standard Conv2D. Train for 15 epochs.

(a) Print the total parameter count for *both* models using `sum(p.numel() for p in model.parameters())`.

(b) Report test accuracy for both. Did the mobile-friendly model maintain comparable accuracy?

(c) Compute the actual parameter compression ratio and compare it with your theoretical calculation from 2A.3.

2B.3 Data Augmentation Study (10 pts).

Real currency notes are crumpled, rotated, and photographed under varying lighting.

(a) Train your mobile-friendly model with the following augmentation pipeline:

- `RandomRotation(`$\pm 30^\circ$`)`
- `ColorJitter(brightness=0.3, contrast=0.3)`
- `RandomHorizontalFlip(p=0.5)`
- `RandomAffine(degrees=0, translate=(0.1, 0.1))`

Report the change in test accuracy compared to training without augmentation.

(b) *Thought question:* Should you apply augmentation to the *test* set as well? Justify in 2–3 sentences.

2B.4 Interpretability with Grad-CAM (10 pts).

Using the `pytorch-grad-cam` library:

(a) Generate Grad-CAM heatmaps for the *final* convolutional layer of your baseline model.

(b) Pick one correctly classified image from *each* of three different denominations (e.g., INR 500, INR 200, INR 2000). Overlay the heatmaps on the original images.

(c) Write 3–4 sentences analyzing: Does the model attend to the denomination numeral, Mahatma Gandhi's portrait, the colour, or the motif on the reverse side? Is this consistent across denominations?

2B.5 Confusion Matrix Analysis (8 pts).

(a) Compute and display the confusion matrix for your best model on the test set (use `sklearn.metrics.confusion_matrix` and plot with `seaborn.heatmap`).

(b) Identify the two most commonly confused denomination pairs. Hypothesize why these denominations are confused (consider colour similarity, size, and visual features).

(c) Suggest one targeted strategy (beyond generic augmentation) that could reduce this specific confusion.

Question 3: Demystifying Transformers – Building a Tiny Language Model

Estimated Time: 8 Hours **Total: 70 Points**

Objective: Build a character-level autoregressive Transformer from scratch. Deeply understand self-attention, positional encoding, the role of Layer Normalization in Transformers, and text generation strategies.

Dataset: A plain-text file of Bollywood movie plots, transliterated Hindi poetry (e.g., Kabir Dohe), or any English-language corpus of $\geq$50 KB. Character-level tokenization ensures a tiny vocabulary ($V < 100$).

Note

You must implement the Transformer **from scratch** using only `torch.nn.Linear`, `torch.nn.LayerNorm`, `torch.nn.Embedding`, and basic tensor operations. Do **not** use `torch.nn.TransformerDecoder` or HuggingFace.

Part A: Theoretical Foundations [25 Points]

3A.1 Scaled Dot-Product Attention (5 pts).

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

(a) Explain why we divide by $\sqrt{d_k}$. Specifically, if q and k are random vectors with entries drawn i.i.d. from $\mathcal{N}(0, 1)$, what is the variance of the dot product $q \cdot k$? How does this affect the softmax output?

(b) What would happen to the gradient of the softmax if we omitted the scaling and $d_k = 512$?

3A.2 Causal Masking (5 pts).

(a) Write the 4×4 causal (lower-triangular) attention mask used for a sequence of length 4. Use 0 for allowed positions and $-\infty$ for blocked positions.

(b) Explain why this mask is necessary for autoregressive (left-to-right) language modeling but is *not* used in BERT-style models.

3A.3 Positional Encoding (5 pts).

(a) The original Transformer uses fixed sinusoidal positional encodings. Write the formula for the $(\text{pos}, 2i)$ and $(\text{pos}, 2i+1)$ entries.

(b) Modern models like GPT-2 and LLaMA use *learned* positional embeddings instead. Give one advantage and one disadvantage of learned embeddings compared to sinusoidal ones.

3A.4 Why LayerNorm in Transformers? (6 pts).

(a) **Contrast with BatchNorm (3 pts).** Suppose you have a batch of 4 sequences, each of length 10, with embedding dimension 64 (tensor shape: $4 \times 10 \times 64$). BatchNorm would normalize across the batch dimension (axis 0) for each feature. LayerNorm normalizes across the feature dimension (axis 2) for each token independently. Explain why LayerNorm is preferred when sequences in the batch have been padded to different true lengths.

(b) **Pre-Norm vs. Post-Norm (3 pts).** In the original Transformer (Vaswani et al., 2017), LayerNorm is applied *after* the residual addition (Post-Norm):

$$x \leftarrow \text{LayerNorm}(x + \text{Sublayer}(x))$$

Modern architectures (GPT-2, LLaMA) apply it *before* (Pre-Norm):

$$x \leftarrow x + \text{Sublayer}(\text{LayerNorm}(x))$$

Explain why Pre-Norm leads to more stable gradients during training, especially for deeper models. (*Hint:* Think about the gradient path through the residual connection.)

3A.5 Multi-Head Attention (4 pts).

(a) If $d_{\text{model}} = 64$ and $n_{\text{heads}} = 4$, what is the dimension of Q, K, V for each head?

(b) Why is multi-head attention more expressive than single-head attention with the same total parameter count? Answer in 2–3 sentences.

Hint

For 3A.1(a), use the fact that $\text{Var}(q \cdot k) = \sum_{j=1}^{d_k} \text{Var}(q_j k_j) = d_k$ when entries are i.i.d. $\mathcal{N}(0, 1)$.

Part B: Practical Implementation [45 Points]

3B.1 Build the Transformer (15 pts).

Implement a **decoder-only** Transformer with the following specifications:

Hyperparameter	Value
d_{model} (embedding dim)	64
n_{heads}	4
n_{layers}	3
Context window	128 characters
Feed-forward hidden dim	$4 \times d_{\text{model}} = 256$
Dropout	0.1

Your implementation must include:

- Character-level embedding + positional embedding (learned).
- Multi-head self-attention with a causal mask.
- Position-wise feed-forward network (two linear layers with GELU activation).
- Residual connections and LayerNorm (use Pre-Norm).

Print the total parameter count of your model.

3B.2 Pre-Norm vs. Post-Norm Training Comparison (10 pts).

Train two versions of your Transformer:

- **Version A:** Pre-LayerNorm (as specified above).
- **Version B:** Post-LayerNorm (apply LayerNorm *after* the residual addition).

Train both for 3,000 iterations (or until convergence) using the Adam optimizer ($\eta = 3 \times 10^{-4}$). Plot their training loss curves on the same graph.

In 3–4 sentences, discuss:

- Which converges faster?
- Which achieves a lower final loss?
- Did either version show training instability (loss spikes)?

3B.3 Text Generation (10 pts).

Write a `generate(model, prompt, max_len=200, temperature=1.0)` function.

(a) Generate 200 characters starting from the prompt `"The hero"` at temperature $T = 1.0$. Print the output.

(b) Generate at $T = 0.2$ (near-greedy) and $T = 1.5$ (high entropy). Print both.

(c) In 3–4 sentences, explain how temperature affects the probability distribution over the next character and the qualitative differences you observe.

3B.4 Attention Visualization (10 pts).

(a) For a short input string (e.g., `"the king"`), extract the 4×4 (or 8×8) attention weight matrix from *one head* of the first layer.

(b) Visualize it as a heatmap using `matplotlib` (x-axis: key characters, y-axis: query characters).

(c) Do the same for one head from the *last* layer.

(d) In 2–3 sentences, compare the two heatmaps. Are early-layer heads more "local" (attending to nearby characters) while later-layer heads are more "global"?

Question 4: Concept Deep-Dive – Research & Explain

Estimated Time: 4–5 Hours **Total: 15 Points**

Note

A hallmark of genuine understanding is the ability to *explain* a concept clearly to someone else. In this question you will research a single focused topic from the course, synthesise what you learn, and produce a short document that could serve as a study resource for your classmates.

Deliverable: A single PDF document of **3–5 paragraphs** (maximum 2 A4 pages), uploaded via the course portal.

Important

When you sign in to the course portal, **one topic will be randomly assigned to you** from the bank of 300 topics. Each student will receive a *unique* topic. You must complete the topic assigned to you—submitting work on a different topic will receive **zero marks**. Upload your completed PDF on the portal itself.

Task Description

Your goal is to research your assigned topic using lecture material, textbooks, and reputable web resources, and then write a clear, self-contained explanation that a fellow SMAI student could use as a study guide.

Your document must include:

(a) **Conceptual Explanation** — Describe the idea in your own words. Build intuition *before* introducing formalism. **[4 marks]**

(b) **Mathematical / Algorithmic Core** — Include at least one of the following: a derivation, an algorithm walkthrough, or a numerical dry-run that demonstrates the mechanics. **[4 marks]**

(c) **Visual Element** — Include at least one diagram, plot, table, or worked example that aids understanding. **[3 marks]**

(d) **Connections** — Briefly relate your topic to at least one other concept from the course. **[2 marks]**

(e) **References** — Cite the sources you consulted (lecture slides, textbook chapters, URLs). **[2 marks]**

Formatting guidelines

- Title your document with your assigned topic number and title.
- Use clear section headings (e.g., *Intuition*, *Derivation*, *Example*, *Connections*).
- Maximum **2 A4 pages**, font size ≥ 11 pt, reasonable margins.
- Submit as a **PDF** named `Q4.pdf` via the course portal.

Points Summary

Question	Theory	Implementation	Total
Q1: Optimizers, Normalization, Memory	25	30	55
Q2: Indian Currency CNN	25	50	75
Q3: Tiny Transformer	25	45	70
Q4: Concept Deep-Dive	15	–	15
Grand Total	**90**	**125**	**215**

Estimated Time Breakdown

Component	Hours
Q1: Theory + Implementation	7
Q2: Theory + Implementation	8
Q3: Theory + Implementation	8
Q4: Research + Writing	4
Report polishing & naming	1
Total	**28**

— End of Assignment 2 —